Log-Based Intrusion Detection System

Final Report — v1.0.0

Mia Alvarez | ID: 030261697

CECS 478- 03

Professor Addington

Repository: github.com/miaalvv/Log-Intrusion-Detection

Release: v1.0.0

Reproducible via: make clean && make up && make demo

1. Problem Statement

Brute-force login attacks are one of the most common threats for modern system, in these attacks the attack attempts to repeatedly guess passwords until they gain access to an account. Once inside, the attacker can steal sensitive data, gain higher privileges, or use the account to move into the system. Although these attacks are common, they are often easy to spot because they leave a clear pattern in log files. Many fail login attempts from the same user in a short amount of time.

The problem this project addresses is the sheer lack of an automated too for automatically analyzing authentication logs to detect this kind of behavior. Although tools like Snort and Suricate exist, they are complex and require more setup and infrastructure than needed for simple log analysis. In this project we demonstrate a simple and reproducible system that can parse log files, apply threshold-based detection, and export clear results. Something so simple, a security analyst or student can run in a single command.

This system directly addresses the need for a simple, automated way to detect brute-force login attempts in authentication logs. The system takes a structured authentication log file, identifies users whose failed login count exceeds a configurable threshold, generates terminal alerts, and exports results as both a CSV report and a JSON summary, all within a fully containerized, reproducible pipeline.

---

2. System Design

The system is built as a five-stage pipeline. Each stage is implemented as an independent Python module with a single responsibility. This allows each component to be independently testable and replaceable without affecting the rest of the pipeline.

Architecture

The pipeline flows in one direction: from raw log data to structured evidence artifacts.

Ingest → parse → detect → alert → summarize

logs.txt → parser.py → detector.py → alert.py → summarizer.py → results.csv + summary.json

Modules

Module	Responsibility	Key Design Decision
parser.py	Reads logs.txt line, validates format, rejects invalid entries	LOGIN_SUCCESS and LOGIN_FAILED are accepted
detector.py	Counts LOGIN_FAILED events per user using a defaultdict, flags users at threshold	Threshold is set to default=5 (can be set to what the user whats)
alert.py	Prints [ALERT] messages to terminal for each flagged user	Separates output concern from detection logic
summarizer.py	Writes final summary dictionary to artifacts/release/summary.json	It is responsible only for formatting and storing data

Module	Responsibility	Key Design Decision
main.py	Orchestrates the full pipeline, computes metrics, generates results.csv	Controls the overall flow of the program
pcap_reader.py	Loads sample.pcap using scapy, appends packet count to summary	Demonstrates extensibility to network-layer evidence

Protocols

Input logs follow a simple four-field format: date, time, username, and action. If a log does not match the designated format, the parser skips it and prints a [WARN] message. This format ensures that malformed or injected entries never reach the detection engine.

Two action values are recognized, LOGIN_SUCCESS and LOGIN_FAILED. Other values are invalid and rejected at parsing. This input restriction ensures that the detection engine can operate on clean, validated, and structure data.

Outputs are saved in easy-to-use formats, a CSV for the results with labeled columns for user, failed attempt count, flagged status, and summary, and a JSON for the full summary with a pipeline summary, including detection metrics, alert details, and packet count from the network capture.

Design Rationale

This system prioritizes simplicity and modularity, it is easy to run without requiring much setup. Each module has a specific role. The parser is responsible for reading and validating log data ensuring that only properly formatted entries are used. The detection logic focuses on identifying suspicious behavior by applying a threshold rule, specifically looking for multiple failed logins attempts within a short time window. This modular separation keeps the code organized and makes it easier to update or improve individual components without affecting the entire system.

This system also prioritizes transparency through output. Instead of just flagging users as suspicious, the system includes additional details like failed attempt counts, time windows, and the reason for why the user was flagged. This makes the results easier to understand with evidence.

Overall, every system design was made to balance simplicity, clarity, and functionality while demonstrating meaningful intrusion detection concepts.

---

3. Threat Model

Attacker

This system assumes that the authentication logs being analyzed are accurate and have not been altered. It also assumes that each log entry includes consistent fields such as timestamp, username, and action. The system operates on static log files rather than real-time data, meaning detection happens after events have already occurred. Additionally, it assumes that repeated failed login attempts are a reliable indicator of suspicious or malicious behavior.

Attacker Capabilities

The system is designed to detect an external or internal attacker who attempts to gain unauthorized access to a system by repeatedly submitting incorrect credentials like a brute-force attack. They can generate a high number of failed login attempts within a short period of time. However, the attacker cannot modify or delete log files, and their activity is fully recorded in the system logs. The attacker is also assumed to be external to the system and does not already have privileged access.

Trust Boundaries

Component	Trusted?	Rationale
data/logs.txt	Yes	We assume the log file hasn't been changed or tampered with
src/ modules	Yes	All source code is written and controlled by the developer
Docker container	Yes	The container uses a fixed setup and known dependencies
Input log entries	No	Each log entry is treated as untrusted and checked before being used
sample.pcap	Yes	Network capture is a frozen artifact used for demonstration

This trust boundary highlights that while the system relies on external log files, it treats individual log entries as untrusted input and validates them before processing to reduce the risk of malformed or malicious data.

Attacks Prevented and Detected

Attack Type	Prevented / Detected	Mechanism
Brute-force login (single user)	Detected	Failed attempt count >= threshold triggers [ALERT] and CSV flag
Log injection (malformed entries)	Prevented	Parser rejects any line not matching the 4-field format
Unknown action injection	Prevented	Whitelist validation: only LOGIN_SUCCESS and LOGIN_FAILED are accepted
Empty username entries	Prevented	Parser skips entries with empty user field
Slow brute-force (spread over time)	Not detected	Counts are global, not windowed — see Limitations
Credential stuffing (multiple users)	Not detected	Each user is evaluated independently — see Limitations

Risks

One major risk in this system is that it may produce false positives if a legitimate user accidentally enters the wrong password multiple times in a short period. It may also miss more

advanced attacks that do not follow a clear pattern, such as slow brute-force attempts spread over a longer time. Because the system only analyzes static logs, it cannot prevent attacks in real time, only detect them after they have occurred.

---

4. Methods

Implementation

The system was implemented in Python 3.10 using standard library modules such as csv, json, collections, along with scapy for parsing network capture files. The detection logic was intentionally kept simple and easy to understand. This makes the system easier to explain, test, and verify

Detection Algorithm

The detection process works in three main steps:

- Parse: Each long entry is read and then checked to make sure it follows the correct format. Valid entries are then stored in a structured list with fields.
- Count: The system loops through the parsed logs and counts how many failed login attempts each user has, using a dictionary to keep track.
- Flag: After counting, any user who's failed attempts meet or exceeds the threshold is flagged as suspicious

The detection function signature is: detect_suspicious_activity making the threshold configurable without modifying source code.

Evaluation Metrics

To measure how well the system performs, two simple metrics were used:

- Detection Rate (Recall): This shows how many actual suspicious users were correctly identified by the system. Calculated as true_positives / len(ground_truth).
- False Positive Rate: This shows how many users were flagged as suspicious when they were actually not. Calculated as false_positives / (true_positives + false_positives).

These metrics help evaluate whether the system is correctly identifying attacks without over flagging normal behavior.

Experiment Setup

Experiments were tested by running the full pipeline using *make clean && make up && make down* inside a Docker container. The dataset included a mix of normal activity and simulated brute-force login attempts.

The logs observed contained multiple users, with some intentionally set to exceed the failed login threshold to show suspicious behavior. All outputs, including CSV results and JSON summaries, were saved and used as evidence for evaluation. The results were then stored in the artifacts/release/ directory to ensure reproducibility.

---

5. Results

Detection Metrics

Metric	Value	Interpretation
Detection Rate	1.0 (100%)	All 3 known suspicious users were correctly flagged
False Positive Rate	0.0 (0%)	No clean users were incorrectly flagged
Total Log Entries Processed	29	Full dataset ingested with no parsing errors
Suspicious Users Detected	3	user2 (5 fails), user5 (8 fails), user9 (5 fails)
Warning-Level Users	2	user4 (3 fails), user7 (2 fails) — below threshold
Clean Users	5	user1, user3, user6, user8, user10 — zero failures

Per-User Results (results.csv)

User	Failed Attempts	Flagged	Status	Summary
user1	0	False	CLEAN	Logged in successfully with no failures
user2	5	True	SUSPICIOUS	Tried logging in 5 times and failed — flagged
user3	0	False	CLEAN	Logged in successfully with no failures
user4	3	False	CLEAN	Had 3 failed attempt(s) — below threshold
user5	8	True	SUSPICIOUS	Tried logging in 8 times and failed — flagged
user6	0	False	CLEAN	Logged in successfully with no failures
user7	2	False	CLEAN	Had 2 failed attempt(s) — below threshold
user8	0	False	CLEAN	Logged in successfully with no failures
user9	5	True	SUSPICIOUS	Tried logging in 5 times and failed — flagged
user10	0	False	CLEAN	Logged in successfully with no failures

Interpretation

The system performed well On the dataset. All three users that were set to simulate brute force attacks were correctly flagged and no normal users were incorrectly flagged as suspicious. For example, user5 had 8 failed login attempts, the system calls it using the threshold margin. Users at the warning level, like user4 and user7, were not flagged, which confirms that the threshold is working as intended.

However, these results come from a controlled and simulated dataset. In real world setting results would depend on things such as size of the logs, different user behaviors, whether attackers tried to stay within the detection threshold within a certain amount of time. Bring us to our next section on the system's limitations.

---

6. Limitations

This system was designed to detect brute force login attempts from static log files using a simple threshold based approach. Because of this, there are a few limitations:

- **Global counting only:** No time-windowed detection

  Failed login attempts are counted across the entire log file, not within a specific time window. This means someone making one attempt every hour could be flagged the same as someone making multiple attempts in a few seconds. Adding a time window, like 5 attempts within 60 seconds, would make detection more accurate.
- **Static file processing only:** No real-time monitoring

  The system only analyzes logs after they've been created. It doesn't monitor logs live or react to attacks as they happen, so all detection is done after the fact and cannot catch live attacks.
- **Username-only detection:** No IP-based correlation

  Detection is based only on usernames. If an attacker tries logging into many different accounts from the same IP, the system wouldn't catch it because no one user hits the threshold.

---

7. Future Work

There are a few ways this system could be improved to be more realistic and closer to a high level intrusion detection system:

1. **Time-windowed detection**

   Currently, the system counts failed login attempts across the entire log file. A better approach would be to use a time window (for example, 5 failed attempts within 60 seconds). This would help catch slow attackers who spread attempts out over time, while still detecting rapid brute-force attacks.
2. **Real-time log tailing**

   The system currently only analyzes logs after they've been created. Adding real-time monitoring (using Python's watchdog or a tail -f approach) would allow the system to detect and alert on attacks as they happen instead of after they have been logged.